

PROLOG

Phân loại ngôn ngữ

- Tiếp cận lệnh (imperative) : Chương trình là một chuỗi các lệnh
 - Các lệnh điều khiển (if,for,while,...)
 - Các lệnh tính toán (:=,...)
 - Các lệnh vào ra (read,write,print)
 - Phù hợp với nguyên lý của máy tính Von Neumann
 - Ví dụ
 - Assembler
 - Fortran
 - Pascal
 - C
 - ...

Phân loại ngôn ngữ (tiếp)

- Tiếp cận chức năng : Chương trình được thiết kế như một biểu thức toán học. Việc thực thi của chương trình là một phép tính của một biểu thức. Các chức năng được phân rã như những biểu thức con.
 - Phát triển từ lý thuyết λ -calcul của A. Church
 - Ví dụ
 - LISP
 - SCHEME

Phân loại ngôn ngữ (tiếp)

- Tiếp cận hướng đối tượng : Chương trình là một tập các đối tượng. Trong quá trình thực thi, các đối tượng được tạo ra, hủy đi và chúng trao đổi thông tin với nhau qua thông điệp
 - Ví dụ:
 - SmallTalk
 - Eiffel
 - Java
 - C++

Phân loại ngôn ngữ (tiếp)

- Tiếp cận logic : Chương trình là một hệ logic, việc thực thi chương trình là một quá trình lập luận trên hệ logic.
 - PROgrammation en LOGique

Giới thiệu

- Phiên bản đầu tiên ra đời năm 70 ở Marseille
- Tác giả Colmerauer và Kowalski
- Hiện nay, có rất nhiều phiên bản khác nhau của PROLOG
 - Visual PROLOG
 - B-Prolog
 - SWI-Prolog
 - Strawberry Prolog
 - ...

Tiếp cận logic trong Prolog

- Chương trình là tập các mô tả logic vị từ dưới dạng chuẩn HORN
 - Không có cấu trúc điều khiển (rẽ nhánh, lặp)
 - Không có phép gán
- Lập trình bằng cách mô tả tri thức về lĩnh vực
 - Các sự kiện (fact)
 - Các luật (rule)
- Thực thi bằng kỹ thuật chứng minh

Ví dụ

```
child(an,nga).  
child(nga,hung).  
male(an).  
male(hung).  
female(nga).
```

```
father(X,Y) :- child(Y,X),male(X).  
mother(X,Y) :- child(Y,X),female(X).  
grandfather(X,Y) :- father(X,Z),child(Y,Z).
```

```
?- father(hung,nga).  
YES  
?-mother(nga,X).  
X=an  
?-grandfather(_,Y),male(Y).  
Y=an
```


Sự kiện

- Các sự kiện được mô tả thông qua phát biểu lô gic dưới dạng vị từ
 - $\text{pred}(\text{arg}_1, \text{arg}_2, \dots \text{arg}_N)$.
 - pred : tên vị từ (bắt đầu bằng chữ thường)
 - arg_i : tham số vị từ

Tham số vị từ

- Ký hiệu : bắt đầu bằng chữ cái thường hoặc bằng một xâu ký tự
 - book(sodo,vutrongphung)
 - book('Số đỏ','Vũ Trọng Phụng')
- Số
 - age(an,10)
- Biến
 - mortal(X)
- Cấu trúc
 - possess(an,[bycicle,book('Số đỏ','Vũ Trọng Phụng')])

Sự kiện (tiếp)

- Mỗi sự kiện thường mô tả một thuộc tính hoặc một quan hệ nhất định của các tham số vị từ
 - Ví dụ :
 - male(an) : An là đàn ông
 - child(an,nga) : an là con của nga

Sự kiện (tiếp)

- Lưu ý phân biệt giữa khái niệm vị từ và hàm số
 - Hàm : $\text{add}(1,2) \rightarrow 3$
 - Vị từ : $\text{add}(1,2,3)$

Luật

- Một luật trong Prolog là một câu phát triển ở dạng chuẩn Horn
 - $q(\dots) \text{ :- } p_1(\dots), p_2(\dots), \dots, p_n(\dots).$
 - Ví dụ :
 - $\text{where_food}(X, Y) \text{ :- } \text{location}(X, Y), \text{edible}(X).$
 - $\text{mother}(X, Y) \text{ :- } \text{child}(Y, X), \text{female}(X).$
 - Một vài phiên bản Prolog cho phép mô tả những kết nối hoặc, tuy nhiên tránh sử dụng các luật dạng này trong Prolog

Luật (tiếp)

- Luật thường sử dụng để định nghĩa một vị từ (một quan hệ) mới dựa trên các vị từ (quan hệ) đã biết
 - $q(\dots)$ đúng nếu như mọi $p_1(\dots), p_2(\dots), \dots, p_n(\dots)$ là đúng
 - Một vị từ có thể được định nghĩa bằng nhiều luật
 - Ví dụ
 - $\text{sibling}(X, Y) \text{ :- mother}(Z, X), \text{mother}(Z, Y).$
 - $\text{sibling}(X, Y) \text{ :- father}(Z, X), \text{father}(Z, Y).$

Chương trình Prolog

- Một chương trình Prolog là một tập các sự kiện và luật
- Có thể soạn thảo độc lập chương trình Prolog, sau đó nạp vào bằng cách :
 - `?-consult('d:\\test.pro').`

Truy vấn

- Sau khi đã nạp chương trình bằng consult(), tại dấu nhắc ?- có thể thực hiện truy vấn bằng cách đặt ra những câu hỏi

?-male(an).

Yes

Truy vấn

- Trong câu truy vấn có thể đặt ra nhiều điều kiện
?-grandfather(X,Y),male(Y).
X=hung
Y= an
- Sử dụng các biến ẩn danh
?-grandfather(_,X),male(X).
X=an

Ví dụ

Chương trình

```
child(an,nga).  
child(nga,hung).  
male(an).  
male(hung).  
female(nga).
```

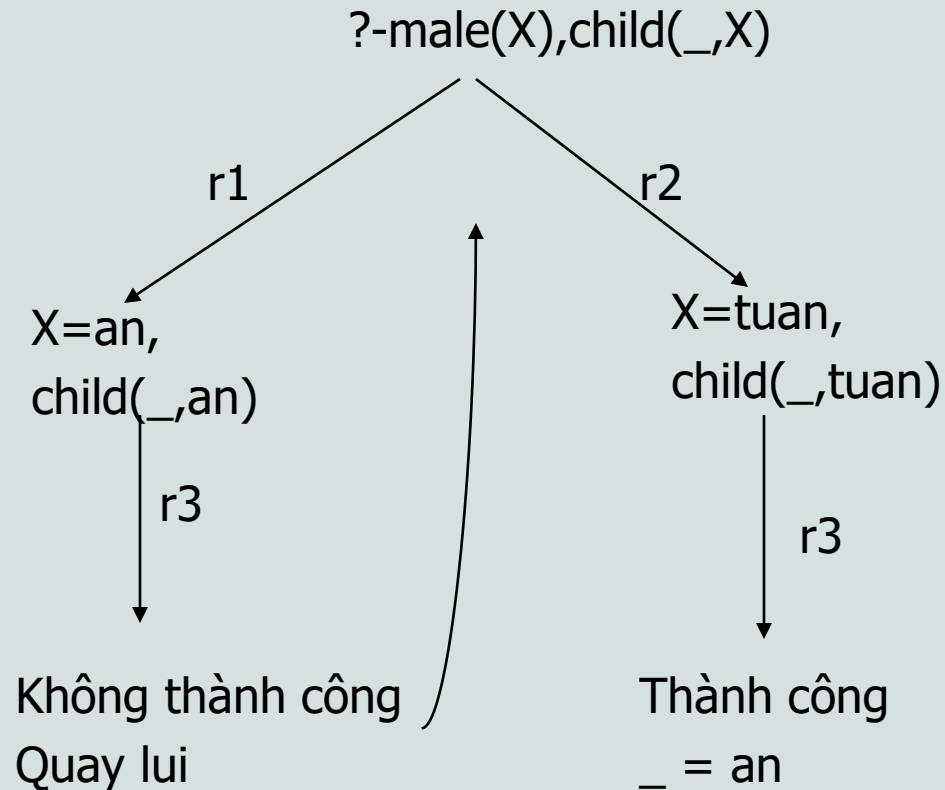
```
father(X,Y) :- child(Y,X),male(X).  
mother(X,Y) :- child(Y,X),female(X).  
grandfather(X,Y) :- father(X,Z),child(Y,Z).
```

Truy vấn

```
?- father(hung,nga).  
YES  
?-mother(nga,X).  
X=an  
?-grandfather(_,Y),male(Y).  
Y=an
```

Cây tìm kiếm

(r1) male(an).
(r2) male(tuan).
(r3) child(an,tuan).



Cây tìm kiếm

- Vấn đề đặt ra
 - Kết quả tìm kiếm phụ thuộc thứ tự sắp xếp các sự kiện và luật
 - Có thể một kết quả bị lặp lại
 - Ví dụ:

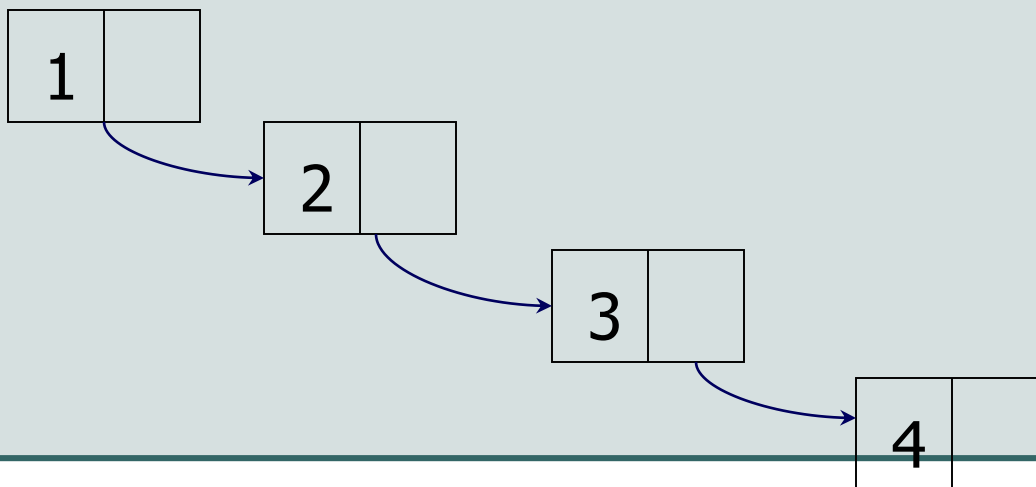
Chương trình

```
child(an,nga).  
child(an,tuan).  
child(hoa,nga).  
child(hoa,tuan).  
male(an).
```

```
brother(X,Y) :- child(X,Z),child(Y,Z),  
                 male(X),\=(X,Y).
```

Danh sách

- Biểu diễn
 - [1,2,3,4]
 - []
 - [1|2|[3|[4|[]]]]
- Thực tế danh sách biến thể từ cây nhị phân



Thao tác với danh sách

- Lấy phần tử đầu tiên của danh sách
 - `car([],[]).`
 - `car([X|_],X).`
- Lấy danh sách tiếp theo
 - `cdr([],[]).`
 - `cdr([_|X],X).`

`?-car([a,b,c],X).`

`X=a`

`?-cdr([a,b,c],X).`

`X=[b,c]`

Thao tác với danh sách

- Kiểm tra thành phần của một danh sách
 - `member(X,[X|_]).`
 - `member(X,[_|L]):-member(X,L).`

`?-member(c,[a,b,c]).`

Yes

`?-member(X,[a,b,c]).`

`X=a`

`X=b`

`X=c`

Áp dụng Prolog trong bài toán chuyển trạng thái

- Sói, dê, bắp cải muốn qua sông
- Người lái đò chỉ có thể chuyển một con vật sang tại một thời điểm
- Sói sẽ ăn dê, dê sẽ ăn bắp cải nếu như không có người lái đò

Trạng thái

- `start_state(left,left,left,left).`
- `end_state(right,right,right,right).`

Phép chuyển

- `oppose(left,right).`
- `oppose(right,left).`
- `transition(state(S1,D1,B1,N1),state(S1,D1,B1,N2)):-
 oppose(N1,N2).`
- `transition(state(N1,D1,B1,N1),state(N2,D1,B1,N2)):-
 oppose(N1,N2).`
- `transition(state(S1,N1,B1,N1),state(S1,N2,B1,N2)):-
 oppose(N1,N2).`
- `transition(state(S1,D1,N1,N1),state(S1,D1,N2,N2)):-
 oppose(N1,N2).`

Kiểm tra trạng thái hợp lệ

- `valid_state(_,D,_,D).`
- `valid_state(F,_,F,F).`

Tìm kiếm

- `solution(S):-start_state(E),findpath(E,[E],S).`
- `findpath(Current_state,Current_path,Solution):-
end_state(Current_state),Solution=Current_path.`
- `findpath(Current_state,Current_path,Solution):-
transition(Current_state,Next_state),
valid_state(Next_state),
notmember(Next_state,Current_state),
findpath(Next_state,[Next_state|Current_path],Solution).`

Toán tử

- Trong PROLOG, toán tử đóng vai trò giống như một cấu trúc chữ không thực hiện tính toán
 - $1+2$ giống $+(1,2)$
 - `hoang yeu van` giống `yeu(hoang,van)`
- Một toán tử có thể được định nghĩa thông qua 4 thuộc tính
 - Tên
 - Vị trí
 - Độ ưu tiên
 - Tính kết hợp

Toán tử (tiếp)

- Vị trí
 - Thứ tự trước : - 5
 - Thứ tự giữa : 3 + 5
 - Thứ tự sau : 3 inv
- Độ ưu tiên:
 - Thứ tự kết hợp trong một cấu trúc phức hợp
 - $2 + 3 * 4 \rightarrow 2 + (3 * 4)$
- Tính kết hợp
 - Kết hợp phải : $x \text{ op } y \text{ op } z \rightarrow (x \text{ op } y) \text{ op } z$
 - Kết hợp trái : $x \text{ op } y \text{ op } z \rightarrow x \text{ op } (y \text{ op } z)$

Toán tử (tiếp)

- Định nghĩa toán tử
 - op(mức ưu tiên,kết hợp,tên)
 - Mức ưu tiên : 1 đến 255
 - Kết hợp :
 - xfx : thứ tự giữa
 - xfy : thứ tự giữa kết hợp trái
 - yfx : thứ tự giữa kết hợp phải
 - fx : thứ tự trước
 - xf : thứ tự sau
 - Tên : phải đặt trong 2 dấu nháy “ với các ký hiệu đặc biệt

Toán tử (tiếp)

- Ví dụ
 - `op(21,yfx, '/')`.
 - `op(21,yfx, '*')`.
 - `op(31,yfx, '+')`.
 - `op(31,yfx, '-')`.
 - `op(11,xfx,mod)`.

Toán tử = và is

- ?- $3 = 1 + 2$.

No

- $X = 1 + 2$.

$X = 1 + 2$

Yes

- ?- $3 \text{ is } 1 + 2$.

Yes

- ?- $X \text{ is } 1 + 2$.

$X = 3$

Tính giai thừa

- $\text{Fact}(0,1).$
- $\text{Fact}(N,F):-$
 - $N1 \text{ is } N - 1$
 - $\text{fact}(N1,F1),$
 - $F \text{ is } F1 * N.$

Phép hợp nhất

- Phép hợp nhất cho phép hợp nhất các biến với các giá trị (ký hiệu, số,...) hoặc thậm chí với các biến khác
- $[a, X, c] = [a, b, Y]$.
 $X = b, Y = C$
- Phép hợp nhất chỉ thực hiện trên sơ đồ cú pháp chứ hoàn toàn không thực hiện thao tác tính toán nào

Phép hợp nhất

- Nếu 2 thành phần hợp nhất là các hằng số thì sẽ thành công khi chúng giống nhau
- Nếu 1 thành phần hợp nhất là biến số , thành phần còn lại là hằng thì biến sẽ nhận giá trị hợp nhất là hằng

Phép hợp nhất

- $f(W, g(X), X, Y) = f(X, Z, 1, g(Z))$.
→ $W = X, g(X) = Z, X = 1, Y = g(Z)$
→ $W = X = 1, g(1) = Z, Y = g(g(1))$
→ $X = 1, W = 1, Z = g(1), Y = g(g(1))$
- $f(g(Y), Y) = f(Z, Z)$. ?
- Toán tử $\neq$: kiểm tra tính không hợp nhất được

Điều khiển quá trình suy diễn

- cut (!)
- fail
- true
- repeat

Cut

- Vị từ thường được biểu diễn bởi dấu ! Được sử dụng để loại bỏ những lựa chọn tiếp tục trong quá trình quay lui

$p(X) :- q(X), !$

$p(a).$

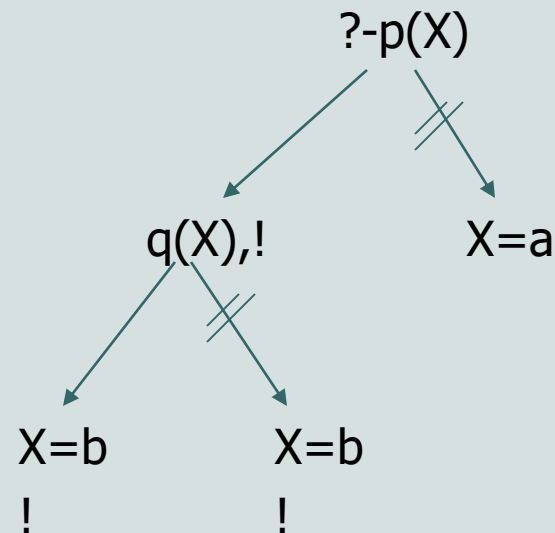
$q(b).$

$q(c).$

$?- p(X).$

b

$yes.$



True

- Mô tả một vị từ luôn đúng
mother(mai,lan) :- true.

Tương đương với
mother(mai,lan).

Fail

- Mô tả một vị từ luôn sai
`live_forever(X) :- fail.`

repeat

- Vị từ cho phép lặp mãi mãi
test :- repeat, write('test'),nl,fail.

Ví dụ

```
member(X,[X|_].  
member(X,[_|L]):-member(X,L)
```

```
?- member(a,[a,b,a]).  
True?  
True?
```

```
member(X,[X|_]):-!.  
member(X,[_|L]):-member(X,L)
```

```
?- member(a,[a,b,a]).  
True  
?member(X,[a,b,a])  
X=a
```

Cut & Fail

- Sử dụng để định nghĩa
 - Tính không hợp nhất được ($\backslash=$)
 - $X \backslash= X \text{ :- } !, \text{fail.}$
 - $X \backslash= Y.$
 - Phủ định
 - $\text{not } p \text{ :- call}(P), !, \text{fail.}$
 - $\text{not } p.$

Thay đổi cơ sở tri thức

- Bổ sung một sự kiện hoặc luật
 - asserta(A)
 - assertz(A)
- Loại bỏ sự kiện hoặc luật
- retract(A)

Vào ra

- Ghi ra màn hình
 - write(exp)
- Đọc từ bàn phím
 - read(E)